

# Data Analysis using Python

## 02 – Introduction to DataFrames

# Contents

---

Anatomy of a DataFrame

Creating DataFrames

Filters

Working with Rows and Columns

Sorting – by values, by index

Multi part indexes

# Anatomy of a DataFrame

The diagram illustrates the anatomy of a DataFrame using a table of basketball player statistics. Annotations include:

- Column names:** A blue label pointing to the header row.
- Columns axis=1:** A grey label pointing to the column headers.
- Index label:** A purple label pointing to the index column.
- Index axis=0:** A grey label pointing to the index values.
- Missing value:** A pink label pointing to the 'NaN' value in the 'Number' column for Jordan Mickey.
- Data:** An orange label pointing to the data cells in the 'Age', 'Weight', and 'Salary' columns.

|   | <i>Name</i>     | <i>Team</i>    | <i>Number</i> | <i>Position</i> | <i>Age</i> | <i>Height</i> | <i>Weight</i> | <i>College</i>    | <i>Salary</i> |
|---|-----------------|----------------|---------------|-----------------|------------|---------------|---------------|-------------------|---------------|
| 0 | Avery Bradley   | Boston Celtics | 0.0           | PG              | 25.0       | 6-2           | 180.0         | Texas             | 7730337.0     |
| 1 | John Holland    | Boston Celtics | 30.0          | SG              | 27.0       | 6-5           | 205.0         | Boston University | NaN           |
| 2 | Jonas Jerebko   | Boston Celtics | 8.0           | PF              | 29.0       | 6-10          | 231.0         | NaN               | 5000000.0     |
| 3 | Jordan Mickey   | Boston Celtics | NaN           | PF              | 21.0       | 6-8           | 235.0         | LSU               | 1170960.0     |
| 4 | Terry Rozier    | Boston Celtics | 12.0          | PG              | 22.0       | 6-2           | 190.0         | Louisville        | 1824360.0     |
| 5 | Jared Sullinger | Boston Celtics | 7.0           | C               | NaN        | 6-9           | 260.0         | Ohio State        | 2569260.0     |
| 6 | Evan Turner     | Boston Celtics | 11.0          | SG              | 27.0       | 6-7           | 220.0         | Ohio State        | 3425510.0     |



# Creating DataFrames

---

There are a few ways to create a DataFrame

The 2 most commons ways are

- Use the **DataFrame()** constructor

- Use one of the **read\_** methods

# Creating DataFrames

The DataFrame Constructor can be used to create

Empty DataFrames

Prepopulated DataFrames

Labeled or unlabeled rows

Labeled or unlabeled columns

any combination of the above

```
df_Vol = pd.DataFrame()
```

Create an empty DataFrame

```
d = np.arange(16).reshape(4,4)  
ndx = ['a', 'b', 'c', 'd']  
cols = ['one', 'two', 'three', 'four']
```

Create a preconfigured and  
populated DataFrame

```
df = pd.DataFrame(data=d, index=ndx, columns=cols)
```

# Creating DataFrames

Pandas has a number of functions that can  
Create a DataFrame **AND**  
Import data stored in various formats into a DataFrame.

All begin with the prefix **read\_**

Each function accepts its own set of arguments and parameters

Create a DataFrame Called **df**

Import the contents of constituents.csv into a DataFrame

```
df = pd.read_csv(filepath_or_buffer='../Data/constituents.csv', index_col='Symbol')
```

Location of csv file

Set the index to be the data in the column '**Symbol**'

The most common are

|                |              |
|----------------|--------------|
| read_csv       | read_orc     |
| read_excel     | read_sas     |
| read_json      | read_stata   |
| read_html      | read_hdf     |
| read_sql       | read_feather |
| read_sql_table | read_gbq     |
| read_sql_query | read_table   |
| read_pickle    | read_fwf     |
| read_clipboard | read_xml     |
| read_parquet   |              |

# Filters

---

## Filter by a Single Condition:

Use Boolean indexing to select rows.

```
df[df['column'] > 10]
```

## Filter by Multiple Conditions:

Combine conditions using & (AND) or | (OR).

```
df[(df['column1'] > 10) & (df['column2'] == 'value')]
```

## Filter with isin:

Select rows where a column's value matches a list of values.

```
df[df['column'].isin(['value1', 'value2'])]
```

## Filter with query:

Use a query string to filter rows.

```
df[df['column'] > 10]
```

# Rows and Columns

## Recall

```
df['Name']
```

```
df[['Name', 'Sector', 'Price']]
```

### Selecting columns

Use `[]` - **SQUARE BRACKETS**

**Single column:** Access a single column by its name.

**Multiple columns:** Select multiple columns using a list of column names.

```
df.loc['FB']
```

```
df.loc[['IBM', 'MMM', 'C']]
```

### Selecting rows – By Index **Label**

Use `.loc[]` - **SQUARE BRACKETS**

**Single row:** Access a single row by its name.

**Multiple rows:** Select multiple rows using a list of rows names.

```
df.iloc[10] # row at position 10
```

```
df.iloc[[10, 20, 30]] rows at positions 10, 20 and 30
```

### Selecting rows – By Index **Position**

Use `.iloc[]` - **SQUARE BRACKETS**

**Single row:** Access a single row by its name.

**Multiple rows:** Select multiple rows using a list of rows names.

# Rows and Columns

---

`[]`, and `.loc[]` can also accept 2 or 3 parameters that can be

Row and Columns

Slices – start:stop

Slices – start:stop:stride

Any combination of the above

## Key Differences

| Method               | Input Type     | Purpose                                  |
|----------------------|----------------|------------------------------------------|
| <code>[]</code>      | Column(s) name | Select columns.                          |
| <code>.iloc[]</code> | Integer index  | Select rows/columns by integer position. |
| <code>.loc[]</code>  | Label index    | Select rows/columns by label.            |

# Rows and Columns

## using []

```
import pandas as pd

# Sample DataFrame
data = {
    'A': [10, 20, 30],
    'B': [40, 50, 60],
    'C': [70, 80, 90]
}
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])

df['A'] # Access column A
df[['A', 'C']] # Subset of columns A and C
```

Single Column

Multiple columns

# Using .loc[] (Label-Based Indexing)

## Using .loc[] (Label-Based Indexing)

```
import pandas as pd

# Sample DataFrame
data = {
    'A': [10, 20, 30],
    'B': [40, 50, 60],
    'C': [70, 80, 90]
}
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])
```

`df.iloc[1]` # Row2

Single row  
(2nd row)

`df.iloc[1, 0]` # Element at Row2, Column A

Single element  
(2nd row, 1st column)

`df.iloc[0:2, 0:2]` # Rows Row1, Row2; Columns A, B

Multiple rows and columns  
(1st and 2nd rows, 1st and 2nd columns)

`df.iloc[[0, 2], [0, 2]]` # Rows Row1, Row3; Columns A, C

Specific rows and columns using lists

# Rows and Columns – using []

## Using .loc[] (Label-Based Indexing)

```
import pandas as pd
```

```
# Sample DataFrame
```

```
data = {  
    'A': [10, 20, 30],  
    'B': [40, 50, 60],  
    'C': [70, 80, 90]  
}
```

```
df.loc['Row2'] # Access Row2
```

```
df.loc['Row2', 'A'] # Element at Row2, Column A
```

```
df.loc[['Row1', 'Row3'], ['A', 'C']] # Rows Row1, Row3; Columns A, C
```

```
df.loc['Row1':'Row2', 'A':'B'] # Rows Row1 to Row2; Columns A to B
```

Single row by label

Single element  
(Row2, Column A)

Multiple rows and columns

Row and column slices

# Removing Rows and Columns

---

## Three Choices, **drop**, **pop** and **del**

### **drop**

Removes rows or columns from a DataFrame based on labels or indices.

Flexible: Can work on rows (**axis=0**) or columns (**axis=1**).

Does not modify the original DataFrame by default; requires **inplace=True** to do so.

### **pop**

Removes a single column from a DataFrame by name.

Returns the removed column as a Pandas Series.

Always modifies the original DataFrame.

### **del**

Deletes a column from a DataFrame in place.

Operates similarly to pop but does not return the deleted column.

Ideal for quick column removal without needing the removed data.

# Removing Rows and Columns – **drop**

```
import pandas as pd
```

## Removing **Rows** using drop

```
# Create a sample DataFrame
```

```
data = {  
    'A': [1, 2, 3],  
    'B': [4, 5, 6],  
    'C': [7, 8, 9]  
}
```

```
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])
```

```
df.drop('Row1', axis=0) # Removes Row1
```

```
df.drop(['Row1', 'Row2'], axis=0) # Removes Row1 and Row2
```

```
df.drop(df.index[0], axis=0) # Removes the first row
```

Original DataFrame

Drop a single row by label:

Drop multiple rows by label:

Drop a row by index:

# Removing Rows and Columns - **drop**

```
import pandas as pd
```

Removing **Columns** using drop

```
# Create a sample DataFrame
```

```
data = {
```

```
    'A': [1, 2, 3],
```

```
    'B': [4, 5, 6],
```

```
    'C': [7, 8, 9]
```

```
}
```

```
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])
```

```
df.drop('B', axis=1) # Removes column B
```

```
df.drop(['A', 'C'], axis=1) # Removes columns A and C
```

```
df.drop('Row1', axis=0, inplace=True) # Removes Row1 in place
```

Original DataFrame

Drop a single column by label:

Drop multiple columns by label:

To modify the DataFrame directly without creating a new one, use **inplace=True**:

# Removing Columns - **pop**

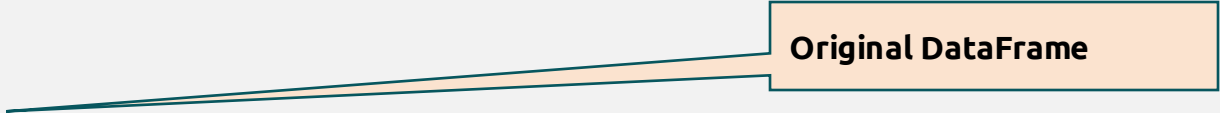
```
import pandas as pd
```

```
# Create a sample DataFrame
```

```
data = {  
    'A': [1, 2, 3],  
    'B': [4, 5, 6],  
    'C': [7, 8, 9]  
}
```

```
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])
```

```
popped_column = df.pop('B') # Removes column 'B' and stores it in 'popped_column'
```



Original DataFrame

## Pop Multiple Columns

**pop** does not natively support multiple columns.

To remove multiple columns, you would need to use a loop or another method like drop.

## Pop a Single Column

Remove a column and store it in a variable:

# Removing Columns - **del**

```
import pandas as pd
```

```
# Create a sample DataFrame
```

```
data = {
```

```
    'A': [1, 2, 3],
```

```
    'B': [4, 5, 6],
```

```
    'C': [7, 8, 9]
```

```
}
```

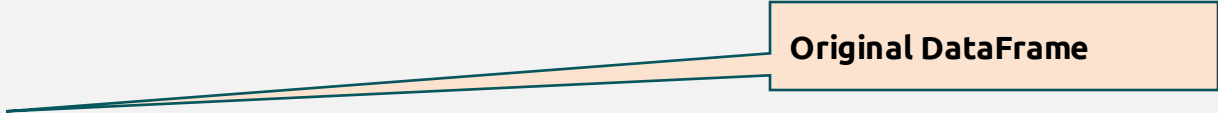
```
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])
```

```
del df['B'] # Removes column 'B'
```

## Delete Multiple Columns

**del** does not natively support removing multiple columns at once. You would need to use **del** in a loop or use another method like **drop**.

To remove multiple columns, you would need to use a loop or another method like **drop**.



Original DataFrame

## Delete a Single Column

Remove a column in place:

# drop, pop and del – Key differences

| Method | Scope                   | Returns                    | Modifies-in-Place        | Works On                          | Key Use                             |
|--------|-------------------------|----------------------------|--------------------------|-----------------------------------|-------------------------------------|
| drop   | removes rows/columns    | New DataFrame (default)    | No (unless inplace=True) | Rows (axis=0)<br>Columns (axis=1) | Selectively remove rows or columns. |
| pop    | Removes a single column | Removed column as a Series | Yes                      | Columns only                      | Remove a column and keep its data.  |
| del    | Deletes a single column | None                       | Yes                      | Columns only                      | Quickly delete a column.            |

# sort\_values()

---

## What is sort\_values?

A Pandas method used to sort a DataFrame or Series by one or more columns or index.

Can perform sorting in ascending or descending order.

Supports custom sorting logic through various options.

## Key Features:

Sorts by column(s) or index.

Handles missing values (**NaN**) gracefully.

Supports sorting in-place or returning a sorted copy.

# sort\_values()

---

## Basic Syntax:

```
df.sort_values(by, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last')
```

## Important Parameters:

### by

Column(s) or index to sort by.

Accepts a string (single column) or list of strings (multiple columns).

### axis

Specifies whether to sort by rows (**axis=0**) or columns (**axis=1**).

Default is 0.

### ascending

Boolean or list specifying sort order for each column.

Default is **True** (ascending order).

### inplace

If True, modifies the DataFrame in place.

Default is False.

### na\_position

Specifies the position of **NaN** values: '**first**' or '**last**'.

Default is 'last'.

# sort\_values()

---

## Sorting by a Single Column:

Sort rows based on the values of a specific column.

Default behaviour is ascending order.

## Sorting by Multiple Columns:

Specify a list of columns in the `by` parameter.

Use a corresponding list for the `ascending` parameter to control sort order for each column.

## Handling Missing Values:

Use `na_position='first'` to bring **NaN** values to the top.

Supported algorithms include `quicksort`, `mergesort`, and `heapsort`.

Use the `kind` parameter to specify the sorting algorithm.

# sort\_index()

---

## What is sort\_index?

A Pandas method used to sort a DataFrame or Series by its **index labels**.

Similar to **sort\_values**, but focuses on sorting by **row index (axis=0)** or **column index (axis=1)**.

## Key Features:

Sorts the DataFrame or Series by index labels in ascending or descending order.

Can sort by **rows (axis=0)** or **columns (axis=1)**.

Handles missing index values (**NaN**) gracefully.

Supports in-place sorting or creating a sorted copy

# sort\_index()

---

```
df.sort_index(axis=0, level=None, ascending=True, inplace=False,  
              kind='quicksort', na_position='last', sort_remaining=True)
```

## Important Parameters:

### axis

Sorts by **row index** (**axis=0**) or **column index** (**axis=1**).

Default is **0**.

### level

Used for sorting multi-index DataFrames or Series.

### ascending

Sort in **ascending** (**True**) or **descending** (**False**) order.

Default is **True**.

### inplace

Modifies the original DataFrame if **True**.

Default is **False**.

# Multi-Indexes

---

A **Multi-Index** (hierarchical index) allows multiple levels of indexing in a Pandas DataFrame or Series.

Enables advanced data manipulation for structured or nested data.

## Key Features

- Supports multi-dimensional data representation.

- Allows hierarchical organization of row or column labels.

- Simplifies grouping, slicing, and aggregation in complex datasets.

## Use Cases

- Time series data (e.g., year, month, day as separate index levels).

- Grouped or nested data (e.g., sales by region and product).

# Multi-Indexes - Creating

---

## From Lists or Arrays:

Provide multiple arrays to define index levels.

```
pd.MultiIndex.from_arrays([array1, array2])
```

## From Tuples:

Use a list of tuples to define hierarchical levels.

```
pd.MultiIndex.from_tuples([(level1, level2), ...])
```

## From Product of Levels:

Generate all combinations of index levels.

```
pd.MultiIndex.from_product([list1, list2])
```

## Set Index Directly:

Convert existing DataFrame columns into Multi-Index.

```
df.set_index(['level1', 'level2'])
```

# Multi-Indexes – Accessing Rows and Columns

---

## Using `.loc[]`:

Access data at specific levels of the index.

```
df.loc[('level1_value', 'level2_value')]
```

## Using `.xs()`:

```
df.xs('level_value', level='level_name')
```

## Slicing:

Use **`slice()`** for ranges in Multi-Index levels.

```
df.loc[(slice(None), 'level2_value')]
```

# Multi-Indexes – Key Operations

---

## Resetting Index:

Convert Multi-Index back to columns.

**`df.reset_index()`**

## Sorting by Levels:

Sort by specific index levels.

**`df.sort_index(level=['level1', 'level2'])`**

